Task Scheduling in Linux / RHEL

Cron, At, Anacron & systemd Timers — Definitions, Use Cases, Examples & Full Command Reference

---

Task scheduling means telling the operating system to run a command or script automatically at a specific time, at regular intervals, or after a delay — without anyone manually triggering it. On RHEL and other Linux systems this is handled by four main mechanisms: **cron** (recurring jobs), **at** (one-time jobs), **anacron** (recurring jobs on machines that aren't always on), and **systemd timers** (the modern, systemd-native replacement/companion to cron). This guide covers what each one is, when to use it, and every command/syntax you'll need in real-world system administration and homelab work.

1. Cron

Definition: Cron is a time-based job scheduler daemon (`crond`) that runs commands automatically at fixed times, dates, or intervals defined in a 'crontab' (cron table) file. It has run in the background on Unix/Linux systems for decades and is the most widely used scheduling tool.

Use case: Recurring maintenance tasks — nightly backups, log rotation, periodic report generation, database cleanup, syncing files, sending reminder emails, or running any script that needs to fire repeatedly on a fixed schedule without a human present.

Cron Syntax

```
* * * * * command_to_run
| | | | |
| | | | +---- Day of week (0-7)  (0 and 7 = Sunday)
| | | +------ Month (1-12)
| | +-------- Day of month (1-31)
| +---------- Hour (0-23)
+------------ Minute (0-59)
```

Special characters: `*` = every value, `,` = list of values (e.g. 1,15), `-` = range (e.g. 1-5), `/` = step values (e.g. */10 = every 10 units).

Managing Crontabs

crontab -e

What it is: Opens the current user's personal crontab file in an editor.

Use case: Adding, editing, or removing your own scheduled jobs.

```
crontab -e
```

crontab -l

What it is: Lists all cron jobs currently scheduled for the logged-in user.

Use case: Checking what jobs are already scheduled before adding a new one.

```
crontab -l
```

crontab -r

What it is: Removes ALL cron jobs for the current user (no confirmation prompt).

Use case: Clearing out a user's entire schedule, e.g. when decommissioning an account.

```
crontab -r
```

crontab -u username -e

What it is: Edits another user's crontab (requires root/sudo privileges).

Use case: An admin managing scheduled jobs for a service account instead of their own login.

```
sudo crontab -u appuser -e
```

Example Cron Jobs

Run a backup every night at 2:30 AM

What it is: Fires the backup script once, daily, at a fixed off-peak time.

Use case: Standard nightly backup automation on a home/production server.

```
30 2 * * * /home/user/scripts/backup.sh >> /var/log/backup.log 2>&1
```

Run every 5 minutes

What it is: Uses a step value to repeat a job at short, fixed intervals.

Use case: Frequent health checks, polling a queue, or syncing a small data feed.

```
*/5 * * * * /home/user/scripts/healthcheck.sh
```

Run only on weekdays at 9 AM

What it is: Restricts execution to Monday through Friday.

Use case: Sending a daily business report only on working days.

```
0 9 * * 1-5 /home/user/scripts/daily_report.sh
```

Run on the 1st of every month

What it is: Fires once a month, on a specific day-of-month.

Use case: Monthly invoice generation or monthly log archiving.

```
0 0 1 * * /home/user/scripts/monthly_archive.sh
```

Run at system reboot

What it is: The @reboot special string runs the job once, every time the system boots up.

Use case: Auto-starting a monitoring script or restoring a service state after a reboot.

```
@reboot /home/user/scripts/startup_check.sh
```

Cron Special Strings (shortcuts)

Shortcut	Equivalent to	Meaning
`@reboot`	—	Run once at startup
`@yearly / @annually`	0 0 1 1 *	Run once a year
`@monthly`	0 0 1 * *	Run once a month
`@weekly`	0 0 * * 0	Run once a week
`@daily / @midnight`	0 0 * * *	Run once a day
`@hourly`	0 * * * *	Run once an hour

System-wide Cron Locations

Location	Purpose
`/etc/crontab`	System-wide crontab (includes a username field)
`/etc/cron.d/`	Drop-in cron files, often installed by packages/services
`/etc/cron.hourly/`	Scripts here run automatically every hour
`/etc/cron.daily/`	Scripts here run automatically every day
`/etc/cron.weekly/`	Scripts here run automatically every week
`/etc/cron.monthly/`	Scripts here run automatically every month
`/var/spool/cron/`	Where individual users' crontabs are actually stored

Checking the cron service on RHEL: `systemctl status crond` and `systemctl enable --now crond`.

2. AT — One-Time Scheduling

Definition: The `at` command schedules a command or script to run exactly ONCE at a specific future time. Unlike cron, it's not for recurring jobs — it's for a single, one-off future execution.

Use case: Running a one-time task like rebooting a server during a planned maintenance window, sending a one-off reminder, or triggering a single deployment step later tonight — without needing to write and then delete a cron entry.

at 10:00 PM

What it is: Opens an interactive at> prompt where you type the command(s) to run once, at the given time, then press Ctrl+D to save.

Use case: Scheduling a single maintenance script for later the same day.

```
at 10:00 PM
at> /home/user/scripts/restart_service.sh
at> <EOT> (Ctrl+D)
```

echo 'command' | at TIME

What it is: Pipes a single command directly into at instead of using the interactive prompt.

Use case: Scripting a one-time scheduled task from another automation script.

```
echo "/home/user/scripts/cleanup.sh" | at 23:45
```

at now + 10 minutes

What it is: Schedules a job relative to the current time instead of an absolute clock time.

Use case: Delaying a task by a short, relative amount, e.g. after a deployment finishes.

```
echo "/home/user/scripts/notify.sh" | at now + 10 minutes
```

atq

What it is: Lists all pending jobs scheduled with at (the 'at queue').

Use case: Checking what one-time jobs are still waiting to run.

```
atq
```

atrm job_number

What it is: Removes/cancels a pending at job using its job number from atq.

Use case: Cancelling a scheduled one-time task before it runs.

```
atrm 3
```

On RHEL, the at daemon service is called `atd` — check it with `systemctl status atd`.

3. Anacron

Definition: Anacron is a scheduler for recurring jobs on machines that are NOT running 24/7 (like a laptop or a homelab machine that gets powered off). Regular cron simply skips a job if the system was off at the scheduled time — anacron instead runs any missed jobs as soon as the machine is back on.

Use case: Daily/weekly/monthly maintenance tasks on a personal machine or homelab host that isn't always powered on, ensuring jobs like log cleanup or backups still happen even if the exact scheduled time was missed.

/etc/anacrontab

What it is: The main config file defining anacron's periodic jobs (period in days, delay in minutes, job identifier, command).

Use case: Defining a job that must run daily regardless of exact uptime windows.

```
# period  delay  job-id       command
1         5      cron.daily   run-parts /etc/cron.daily
```

anacron -T

What it is: Tests the anacrontab file for syntax errors without actually running jobs.

Use case: Validating an anacrontab edit before it goes live.

```
anacron -T
```

anacron -f

What it is: Forces all jobs to run now, ignoring their timestamp records.

Use case: Manually forcing missed or scheduled maintenance jobs to run immediately.

```
sudo anacron -f
```

In practice, RHEL's `/etc/cron.daily`, `cron.weekly`, and `cron.monthly` directories are already wired up to run via anacron, so daily maintenance scripts dropped there benefit from anacron's catch-up behavior automatically.

4. systemd Timers

Definition: systemd timers are the modern, systemd-native way to schedule tasks. A `.timer` unit defines WHEN to run something, paired with a matching `.service` unit that defines WHAT to run. Unlike cron, every run is automatically logged via journald, timers can depend on other services, and missed runs can be caught up automatically.

Use case: Any scheduled task where you want proper logging, the ability to catch up on a missed run after downtime, or a dependency on another service being ready first — common for backups, package metadata refresh, and log rotation on modern RHEL systems (RHEL 7+).

Service unit (defines the task)

What it is: A .service file describing the command to execute.

Use case: Defining what a backup timer should actually run.

```
# /etc/systemd/system/backup.service
[Unit]
Description=Run backup script

[Service]
Type=oneshot
ExecStart=/home/user/scripts/backup.sh
```

Timer unit (defines the schedule)

What it is: A .timer file describing when the paired .service should fire.

Use case: Scheduling the backup.service to run daily at 2:30 AM, with catch-up if missed.

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Run backup daily at 2:30 AM

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
```

systemctl enable --now backup.timer

What it is: Enables the timer to start at boot AND starts it immediately.

Use case: Activating a newly written timer without needing a reboot.

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
```

systemctl list-timers

What it is: Lists all active timers on the system along with their next scheduled run time.

Use case: Checking what's scheduled and when it will next fire, across the whole system.

```
systemctl list-timers --all
```

journalctl -u backup.service

What it is: Views the logs for every past run of the paired service — success, failure, and output.

Use case: Debugging why a scheduled backup didn't complete, with full run history.

```
journalctl -u backup.service --since today
```

OnCalendar Examples

OnCalendar value	Meaning
`*-*-* 02:30:00`	Every day at 2:30 AM
`Mon..Fri 09:00:00`	Every weekday at 9:00 AM
`*:00/15`	Every 15 minutes
`weekly`	Once a week (Monday, 00:00)
`daily`	Once a day (00:00)
`*-*-01 00:00:00`	First day of every month, midnight

5. Comparison — Which One to Use

Tool	Best for	Recurring?	Logging	Catches up if missed?
`cron`	Simple, quick recurring jobs	Yes	Manual (redirect output)	No
`at`	One-time future task	No (once only)	Manual	N/A
`anacron`	Recurring jobs on non-24/7 machines	Yes	Manual	Yes
`systemd timer`	Robust recurring jobs w/ logging & deps	Yes	Automatic (journalctl)	Yes (Persistent=true)

Quick Decision Guide

- Need something to run repeatedly and simply — cron.
- Need something to run exactly once, later today — at.
- Machine isn't always powered on but jobs must still catch up — anacron.
- Need logging, dependency handling, or missed-run catch-up on a modern RHEL box — systemd timer.